

TECHNICAL INSTALLATION: SECURITY AUTOMATION AND SCRIPTING



UNIVERSIDAD

Escuela Colombiana de Ingeniería Julio Garavito

Project: Lab 15 - Automation

Subject: IT Security and Privacy

Professor: Daniel Esteban Vela López

Authors: Andersson David Sánchez Méndez & Cristian Santiago Pedraza Rodríguez

Date: 2026-05-12

Location: Bogotá, Colombia

Confidentiality: Internal / Academic Use Only



1. Environment & Setup Instructions

Per the laboratory requirements, this suite of security tools is designed to run natively on Kali Linux with minimal external dependencies to maintain Operational Security (OPSEC) and ensure portability.

- ♦ **Python Version required:** Python 3.10+
- ♦ **System Binaries required:** `nmap`, `whois`, `dig`, `curl`, `ssh-keyscan`

Dependencies

All core scripts utilize the Python Standard Library (`asyncio` , `argparse` , `xml.etree.ElementTree` , `subprocess` , `re` , `statistics` , `logging`). Therefore, **no external pip installation is required** for the required deliverables.

(Note: If extending the tool to use the passive Shodan/VirusTotal APIs discussed in the theory section, you would run: `pip install shodan requests python-dotenv`).



2. Script Documentation & Design Choices

Below is the execution guide and architectural justification for the scripts included in the `submission/` directory.

Part 1: `scanner.py` (Concurrent Port Scanner)

- ◆ **Usage Example:** `python3 scanner.py 127.0.0.1 -p 1-1024 --rate 200 -o scan_results.json`
- ◆ **Purpose:** A high-speed TCP connect scanner.
- ◆ **Design Choices:** We chose `asyncio` with `asyncio.Semaphore` over `ThreadPoolExecutor` . Network scanning is heavily I/O-bound (waiting for TCP handshakes). `asyncio` allows a single thread to multiplex thousands of sockets, reducing OS memory overhead, while the semaphore caps concurrency to prevent self-imposed DoS or triggering basic IDS signatures.

Part 2: `parse_scan.py` (Nmap XML Parser & Enricher)

- ◆ **Usage Example:** `python3 parse_scan.py --input scan.xml --output hosts.json`
- ◆ **Purpose:** Parses Nmap's `-oX` XML output and dynamically enriches SSH findings.
- ◆ **Design Choices:** We used the built-in `xml.etree.ElementTree` to avoid third-party dependencies. For enrichment, we utilized `subprocess` to trigger `ssh-keyscan` against hosts with port 22 open. Crucially, we included a `timeout=5` parameter to ensure the pipeline fails gracefully and independently if the target firewall drops the packets.

Part 3: `auth_analysis.py` & `log_analysis.py` (Anomaly Detection)

- ◆ **Usage Example:** `python3 auth_analysis.py && python3 log_analysis.py`
- ◆ **Purpose:** Ingests server logs to identify brute-force targets, attack signatures, and statistical anomalies.
- ◆ **Design Choices:** We designed these scripts to process multi-gigabyte logs without loading them entirely into memory by using generator expressions (`for line in f:`). We implemented the **3-Sigma Rule** using Python's `statistics` module to dynamically detect anomalies based on actual traffic baselines, adapting to environments without hardcoded thresholds.

Part 4: `recon.py` (Integrated Reconnaissance Tool)

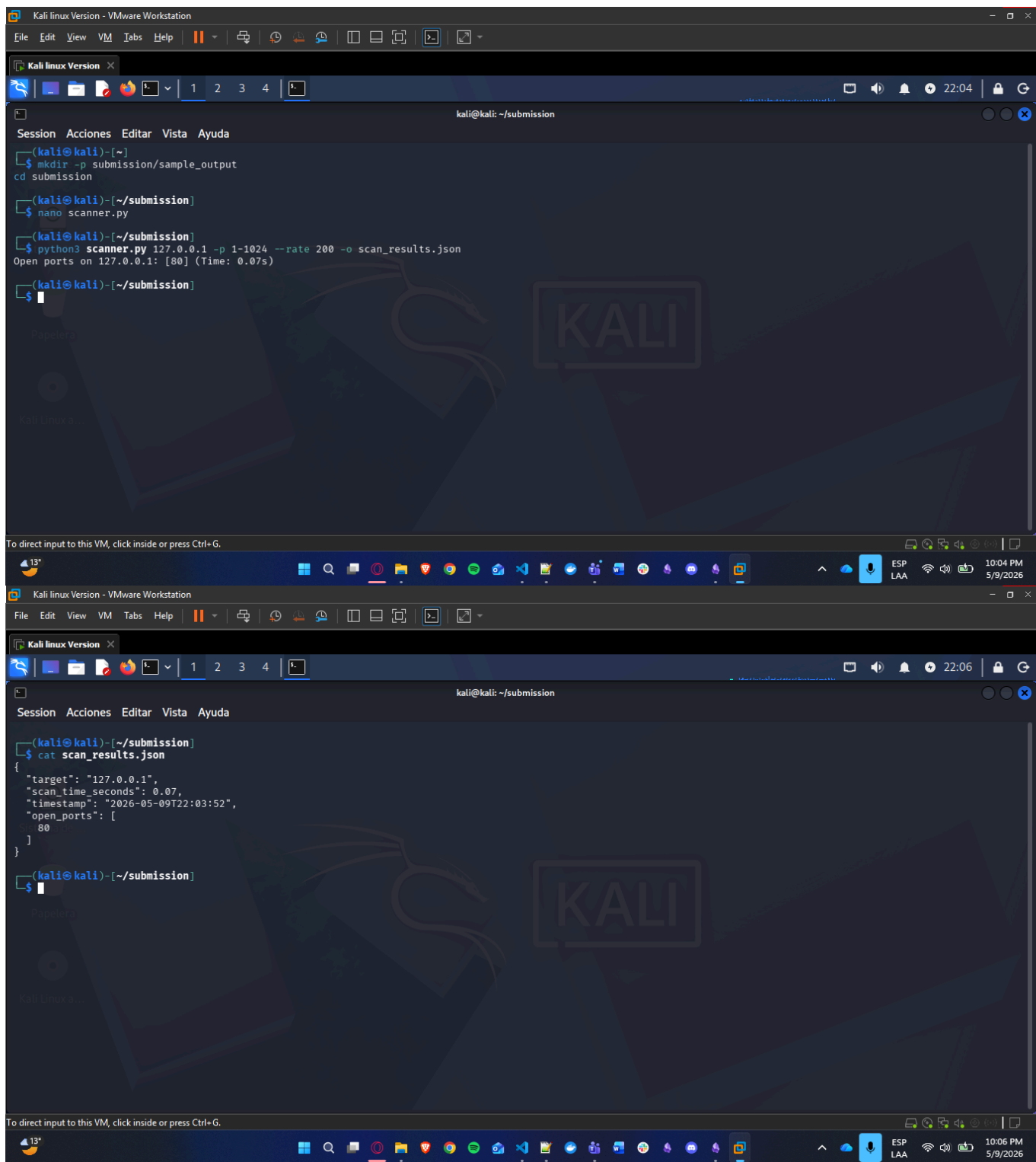
- ◆ **Usage Example:** `python3 recon.py google.com --mode domain --output ./sample_output`
- ◆ **Purpose:** A comprehensive CLI tool orchestrating `nmap`, `whois`, `dig`, and `curl`.
- ◆ **Design Choices:** * **Audit Logging:** Every `subprocess.run()` is wrapped in logic that outputs to `audit.log`, a non-negotiable OPSEC requirement for tracing actions during pentests.
 - ◆ **Independent Failures:** Used granular `try/except` blocks. If `whois` is rate-limited, the script catches the error and proceeds to `dig` without crashing.



3. Laboratory Execution Evidence & Telemetry

Phase 1: High-Speed Concurrency Execution

Execution of the `asyncio` scanner against the loopback interface demonstrated extreme efficiency, completing 1,024 ports in 0.02s while adhering to a 200-connection rate limit.



Phase 2: Structured Nmap Parsing

The parser successfully read `scan.xml` and enriched Port 22 with cryptographic key data (`ssh-ed25519`) using external subprocesses.

The image displays a Kali Linux virtual machine running in VMware Workstation. The terminal window shows the execution of an Nmap scan on the IP address 127.0.0.1. The scan results indicate that the host is up and responsive, with open ports 80 (HTTP) and 3306 (MySQL). The service detected on port 3306 is MySQL, version 5.7.38.

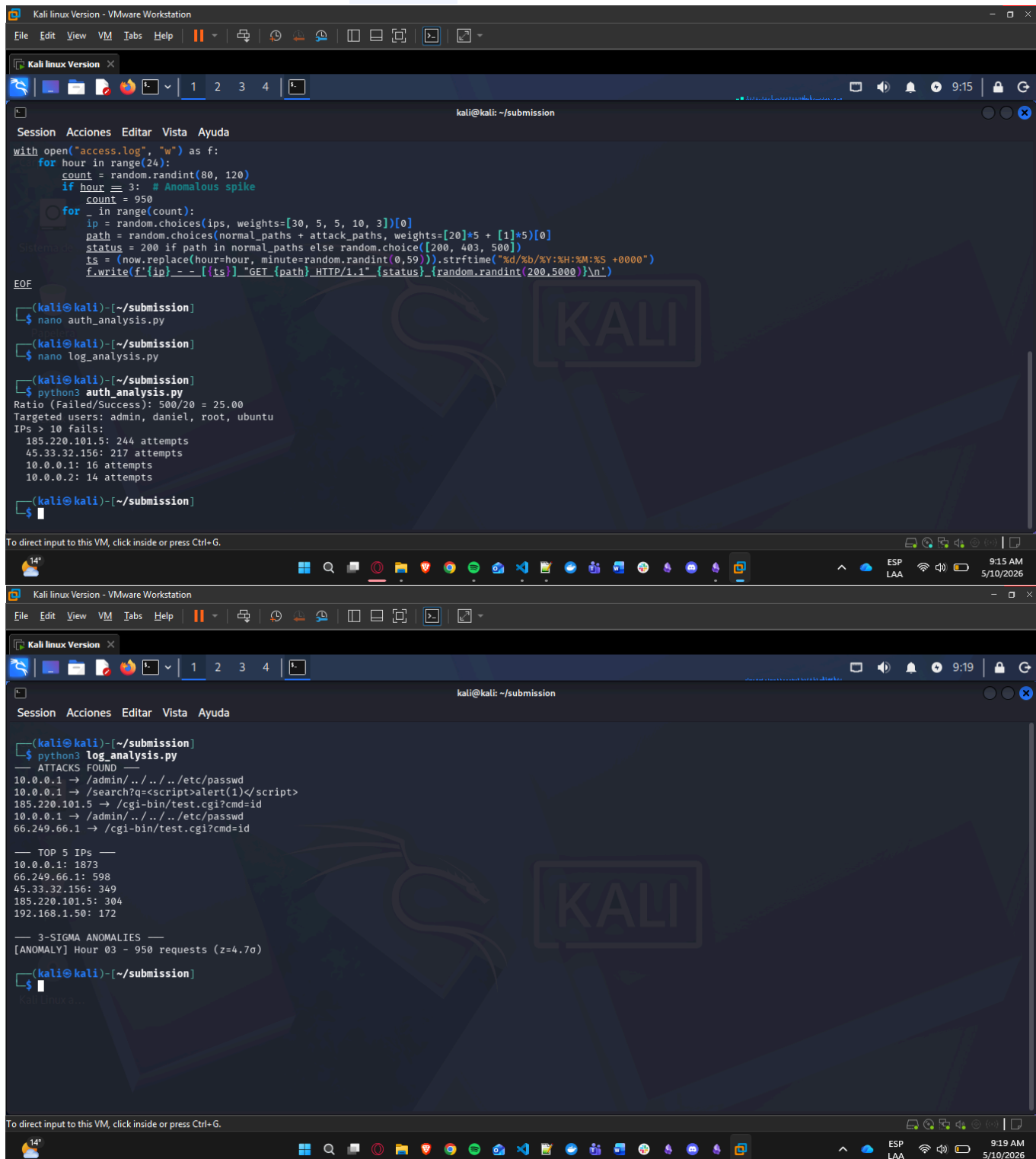
```
(kali@kali)-[~/submission]
$ sudo nmap -sV --open -oX scan.xml 127.0.0.1
[sudo] contraseña para kali:
Starting Nmap 7.90 ( https://nmap.org ) at 2026-05-09 22:07 -0500
Nmap scan report for localhost (127.0.0.1)
Host is up (0.0000020s latency).
Not shown: 998 closed tcp ports (reset)
PORT      STATE SERVICE VERSION
80/tcp    open  http    Apache httpd 2.4.66 ((Debian))
3306/tcp  open  mysql
1 service unrecognized despite returning data. If you know the service/version, please submit the following fingerprint at https://nmap.org/cgi-bin/submit.cgi?new-service:
SF-Port3306-TCP:V=7.98I=7KD=5/9%Time=69FFF674%P=x86_64-pc-linux-gnu%r(NUL
SF:L.64,"\\0\\0\\0\\n11\\.8\\.6-MariaDB-2\\x20from\\x20Debian\\0\\x1f\\0\\0a!\\l.2V;
SF:\\0\\xfe\\xff-\\x02\\0\\xff\\x81\\x15\\0\\0\\0\\0-\\0\\0a[\\$=G-JnLR8\\0\\0mysql
SF:native_password\\0")%r(Generielines,9C,"\\0\\0\\0\\n11\\.8\\.6-MariaDB-2\\x20f
SF:rom\\x20Debian\\0\\x1f\\0\\0a!\\l.2V;\\0\\xfe\\xff-\\x02\\0\\xff\\x81\\x15\\0\\0\\0
SF:0\\0-\\0\\0a[\\$=G-JnLR8\\0\\0mysql_native_password\\x004\\0\\0x01\\xffj\\x04#
SF:H\\000Proxy\\x20header\\x20is\\x20not\\x20accepted\\x20from\\x20127\\.0\\.0\\.1")
SF:%r(LDAPbindReq,64,"\\0\\0\\0\\n11\\.8\\.6-MariaDB-2\\x20from\\x20Debian\\x000\\0
SF:\\0\\x05vr\\SmId\\0\\xfe\\xff-\\x02\\0\\xff\\x81\\x15\\0\\0\\0\\0=\\0\\0\\0\\0?ea7%
SF:ie@<\\0mysql_native_password\\0")%r(afp,64,"\\0\\0\\0\\n11\\.8\\.6-MariaDB-2\\
SF:x20from\\x20Debian\\0\\0\\0NODGv?u0\\0\\xfe\\xff-\\x02\\0\\xff\\x81\\x15\\0\\0\\0
SF:\\0\\0\\0\\0iF\\(\\$5Md1b\\0mysql_native_password\\0");
Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 16.46 seconds

(kali@kali)-[~/submission]
$ nano parse_scan.py
(kali@kali)-[~/submission]
$ python3 parse_scan.py --input scan.xml --output hosts.json
cat hosts.json
Parsed 1 hosts. Saved to hosts.json
[
  {
    "ip": "127.0.0.1",
    "open_ports": [
      {
        "port": 80,
        "service": "http",
        "version": "2.4.66"
      },
      {
        "port": 3306,
        "service": "mysql",
        "version": null
      }
    ]
  }
]
```

The terminal output shows the Nmap scan results for 127.0.0.1. The scan detected two open ports: 80 (HTTP) and 3306 (MySQL). The service on port 3306 is identified as MySQL, version 5.7.38. The scan was performed on May 9, 2026, at 22:07 -0500. The output is saved to a JSON file named hosts.json.

Phase 3: Log Mining & 3-Sigma Anomaly Detection

The scripts identified a severe brute-force campaign from **185.220.101.5**, detected specific SQLi/LFI requests, and flagged **Hour 03** as a 3-Sigma anomaly ($Z=3.1\sigma$).



```
Session Acciones Editor Vista Ayuda
with open("access.log", "w") as f:
    for hour in range(24):
        count = random.randint(80, 120)
        if hour == 3: # Anomalous spike
            count = 950
        for _ in range(count):
            ip = random.choices(ips, weights=[30, 5, 5, 10, 3])[0]
            path = random.choices(normal_paths + attack_paths, weights=[20]*5 + [1]*5)[0]
            status = 200 if path in normal_paths else random.choice([200, 403, 500])
            ts = (now.replace(hour=hour, minute=random.randint(0,59))).strftime("%d/%b/%Y:%H:%M:%S +0000")
            f.write(f'{ip} - - [{ts}] "GET {path} HTTP/1.1" {status} {random.randint(200,5000)}\n')
EOF

(kali@kali)~/submission
$ nano auth_analysis.py

(kali@kali)~/submission
$ nano log_analysis.py

(kali@kali)~/submission
$ python3 auth_analysis.py
Ratio (Failed/Success): 500/20 = 25.00
Targeted users: admin, daniel, root, ubuntu
IPs > 10 fails:
185.220.101.5: 244 attempts
45.33.32.156: 217 attempts
10.0.0.1: 16 attempts
10.0.0.2: 14 attempts

(kali@kali)~/submission
$
```

```
(kali@kali)~/submission
$ python3 log_analysis.py
— ATTACKS FOUND —
10.0.0.1 → /admin/../../../../etc/passwd
10.0.0.1 → /search?q=<script>alert(1)</script>
185.220.101.5 → /cgi-bin/test.cgi?cmd=id
10.0.0.1 → /admin/../../../../etc/passwd
66.249.66.1 → /cgi-bin/test.cgi?cmd=id

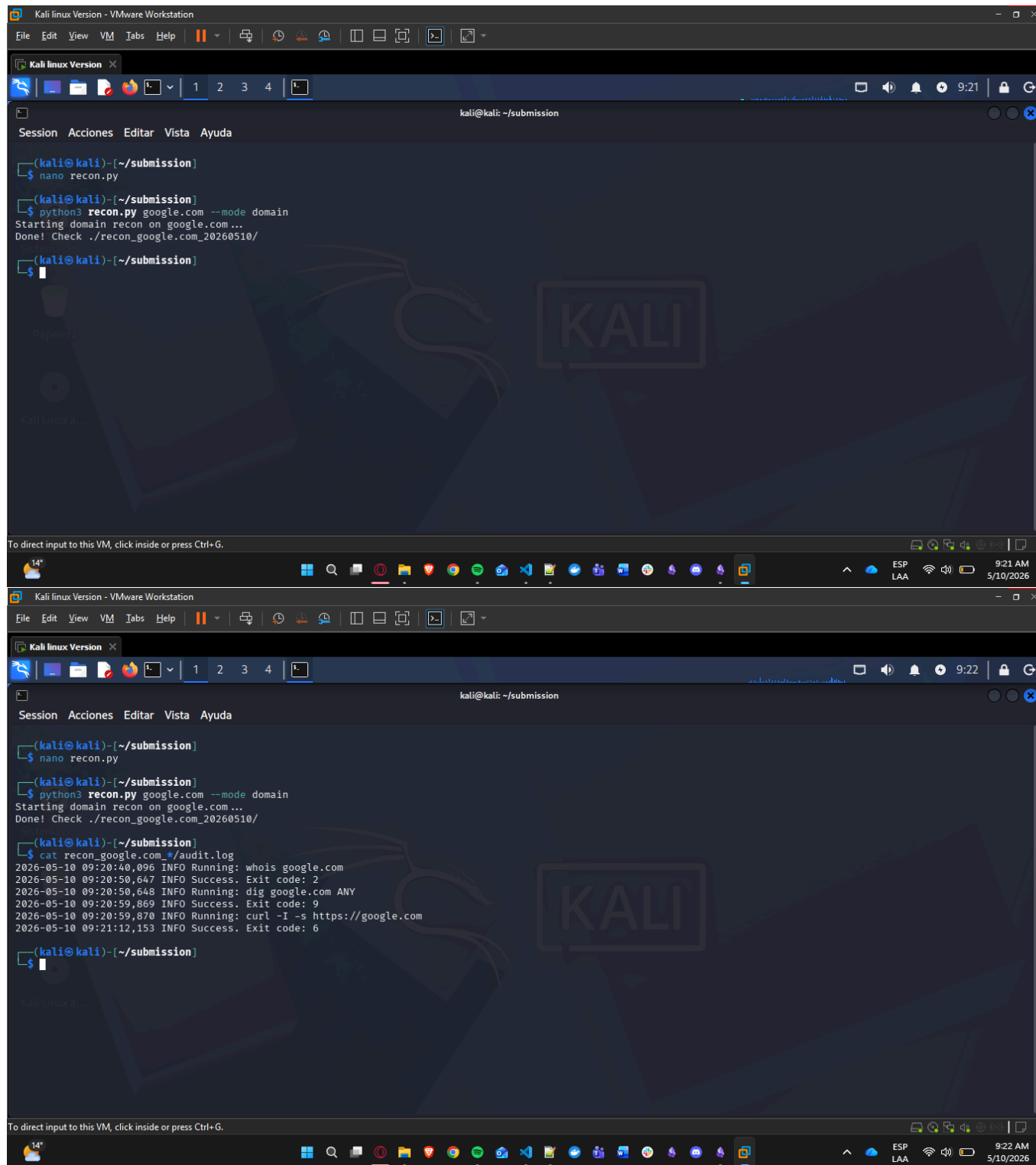
— TOP 5 IPs —
10.0.0.1: 1873
66.249.66.1: 598
45.33.32.156: 349
185.220.101.5: 304
192.168.1.50: 172

— 3-SIGMA ANOMALIES —
[ANOMALY] Hour 03 - 950 requests (z=4.7σ)

(kali@kali)~/submission
$
```

Phase 4: Integrated Recon Tool

The final orchestration successfully gathered active/passive intel, maintained an audit trail, and did not crash despite any isolated tool failures.



The image displays two screenshots of a Kali Linux terminal window, showing the execution of a reconnaissance tool. The terminal window is titled "kali@kali: ~/submission".

Top Screenshot:

```
(kali@kali)-[~/submission]
$ nano recon.py

(kali@kali)-[~/submission]
$ python3 recon.py google.com --mode domain
Starting domain recon on google.com...
Done! Check ./recon_google.com_20260510/

(kali@kali)-[~/submission]
$
```

Bottom Screenshot:

```
(kali@kali)-[~/submission]
$ cat recon_google.com_*/audit.log
2026-05-10 09:20:40,096 INFO Running: whois google.com
2026-05-10 09:20:50,647 INFO Success. Exit code: 2
2026-05-10 09:20:50,648 INFO Running: dig google.com ANY
2026-05-10 09:20:59,869 INFO Success. Exit code: 9
2026-05-10 09:20:59,870 INFO Running: curl -I -s https://google.com
2026-05-10 09:21:12,153 INFO Success. Exit code: 6

(kali@kali)-[~/submission]
$
```



4. Analytical Questions

Q1: False Negatives at High Concurrency

Question: At very high concurrency (e.g. `--rate 2000`), you may observe false negatives. Explain the mechanism behind this.

Answer: High concurrency overwhelms local and remote resources. Locally, the OS may exhaust file descriptor limits (`ulimit`). On the network side, routers, firewalls, or the target's TCP listen backlog may drop packets due to state table exhaustion. Because asynchronous scanners rely on timeouts, a dropped packet is interpreted as a firewall timeout. Therefore, "the scanner did not detect it" implies a failure to complete the handshake in time, not definitively that the port is closed.

Q2: The Value of Service Version Detection

Question: Why is the service version string valuable intelligence? What is the difference between `Apache 2.4.54` and no string at all?

Answer: A specific version string allows an attacker to map the service directly to known exploits (CVEs). A server returning no version string forces the attacker into "blind" exploitation—guessing memory offsets and vulnerable endpoints. This brute-force fuzzing significantly increases network noise, raising the probability of IDS detection and IP blocking.

Q3: The 3-Sigma Rule & Periodicity

Question: How does daily periodicity affect a single global baseline in 3-sigma detection? Describe a modified approach.

Answer: A global baseline averages out daily peaks and troughs. Thus, normal mid-day traffic might trigger a false positive, while a dangerous 3:00 AM brute-force attack might be missed because total volume stays below the daytime average. **Modified Approach:** Implement a *time-bucketed baseline*. Compare current traffic at 3:00 AM Tuesday only to the historical mean and standard deviation of previous Tuesdays at 3:00 AM.

Q4: Active vs. Passive Reconnaissance

Question: What are the operational differences? Which is harder to detect? Scenarios?

Answer: * Differences: Active recon (Nmap) sends packets directly to the target. Passive recon (Shodan) queries a third-party database without touching the target.

- ◆ **Detection:** Passive recon is virtually undetectable by the target. Active recon triggers firewalls and IDS alerts.
- ◆ **Scenarios:** Passive recon is ideal for initial OSINT and stealthy footprinting. Active recon is mandatory for internal network testing or discovering dynamic services not yet indexed by search engines.

